# PARALLEL COMPUTING LAB

BCS702 — VTU 22 Scheme

9 Short, Exam-Friendly Programs

| | |
|---|---|
| **Course** | Parallel Computing |
| **Code** | BCS702 |
| **Scheme** | VTU 22 Scheme |
| **Programs** | 1–9 |
| **Platforms** | OpenMP + MPI |
| **OS** | Ubuntu / Linux |

**Exam note:** These versions are shortened for memorization and lab execution while retaining the required OpenMP/MPI concepts. MPI commands include --oversubscribe for environments such as GitHub Codespaces where slot allocation can be restricted.

# Contents


| | |
|---|---|
| **Program 1** | Parallel and Sequential Merge Sort using OpenMP Sections |
| **Program 2** | OpenMP Static Scheduling |
| **Program 3** | Fibonacci using OpenMP Tasks |
| **Program 4** | Prime Numbers using OpenMP Parallel For |
| **Program 5** | MPI Send and Receive |
| **Program 6** | MPI Deadlock and Avoidance |
| **Program 7** | MPI Broadcast |
| **Program 8** | MPI Scatter and Gather |
| **Program 9** | MPI Reduce and Allreduce |


## Common commands

```
gcc -fopenmp program.c -o program
export OMP_NUM_THREADS=4
./program

mpicc program.c -o program
mpirun --oversubscribe -np 2 ./program
```

# Program 1 – Parallel and Sequential Merge Sort using OpenMP Sections

**Question:** Write an OpenMP program to implement merge sort and compare sequential and parallel execution using OpenMP sections.

**Environment:** Ubuntu/Linux with GCC and OpenMP.

## Code

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void merge(int a[],int l,int m,int r)
{
    int i=l,j=m+1,k=0,t[r-l+1];
    while(i<=m&&j<=r)t[k++]=a[i]<=a[j]?a[i++]:a[j++];
    while(i<=m)t[k++]=a[i++];
    while(j<=r)t[k++]=a[j++];
    for(i=l,k=0;i<=r;i++,k++)a[i]=t[k];
}

void sort(int a[],int l,int r,int p)
{
    if(l<r){
        int m=(l+r)/2;
        if(p){
            #pragma omp parallel sections
            {
                #pragma omp section
                sort(a,l,m,1);
                #pragma omp section
                sort(a,m+1,r,1);
            }
        }else{
            sort(a,l,m,0);
            sort(a,m+1,r,0);
        }
        merge(a,l,m,r);
    }
}

int main()
{
    int n=100000,a[n],b[n];
    double s,e;
    for(int i=0;i<n;i++)a[i]=b[i]=rand()%1000;

    s=omp_get_wtime(); sort(a,0,n-1,0); e=omp_get_wtime();
    printf("Sequential Merge Sort Time: %f seconds\n",e-s);

    s=omp_get_wtime(); sort(b,0,n-1,1); e=omp_get_wtime();
    printf("Parallel Merge Sort Time: %f seconds\n",e-s);
}
```

## Compile

```
gcc -fopenmp prg1.c -o prg1
```

## Run

```
export OMP_NUM_THREADS=4 && ./prg1
```

**Important:** Ubuntu/Linux. The program sorts 100000 random integers and prints sequential and parallel execution times.

# Program 2 – OpenMP Static Scheduling

**Question:** Write an OpenMP program to demonstrate static scheduling with a chunk size of 2.

**Environment:** Ubuntu/Linux with GCC and OpenMP.

## Code

```
#include <stdio.h>
#include <omp.h>

int main()
{
    int n;
    printf("Enter number of iterations: ");
    scanf("%d",&n);

    int t[n];

    #pragma omp parallel for schedule(static,2)
    for(int i=0;i<n;i++) t[i]=omp_get_thread_num();

    for(int i=0;i<n;i+=2)
        printf("Thread %d : Iterations %d -- %d\n",t[i],i,i+1<n?i+1:i);
}
```

## Compile

```
gcc -fopenmp prg2.c -o prg2
```

## Run

```
export OMP_NUM_THREADS=4 && ./prg2
```

**Important:** Ubuntu/Linux. For input 6, iterations are assigned in chunks of two.

# Program 3 – Fibonacci using OpenMP Tasks

**Question:** Write an OpenMP program to generate the Fibonacci series using tasks.

**Environment:** Ubuntu/Linux with GCC and OpenMP.

## Code

```
#include <stdio.h>
#include <omp.h>

int fib(int n)
{
    if(n<2)return n;
    int x,y;
    #pragma omp task shared(x)
    x=fib(n-1);
    #pragma omp task shared(y)
    y=fib(n-2);
    #pragma omp taskwait
    return x+y;
}

int main()
{
    int n;
    printf("Enter number of Fibonacci terms: ");
    scanf("%d",&n);
    printf("Fibonacci Series:\n");

    #pragma omp parallel
    #pragma omp single
    {
        for(int i=0;i<n;i++)
            printf("%d ",fib(i));
    }

    printf("\n");
}
```

## Compile

```
gcc -fopenmp prg3.c -o prg3
```

## Run

```
export OMP_NUM_THREADS=4 && ./prg3
```

**Important:** Ubuntu/Linux. For input 6, output is 0 1 1 2 3 5.

# Program 4 – Prime Numbers using OpenMP Parallel For

**Question:** Write an OpenMP program to find prime numbers from 1 to n and compare serial and parallel execution time.

**Environment:** Ubuntu/Linux with GCC and OpenMP.

## Code

```
#include <stdio.h>
#include <omp.h>

int prime(int n)
{
    if(n<2)return 0;
    for(int i=2;i*i<=n;i++)
        if(n%i==0)return 0;
    return 1;
}

int main()
{
    int n;
    printf("Enter the value of n: ");
    scanf("%d",&n);

    int a[n+1],b[n+1];
    double s,e;

    printf("\nPrime numbers from 1 to %d:\n",n);
    for(int i=1;i<=n;i++)a[i]=prime(i);
    for(int i=1;i<=n;i++)if(a[i])printf("%d ",i);
    printf("\n");

    s=omp_get_wtime();
    for(int i=1;i<=n;i++)b[i]=prime(i);
    e=omp_get_wtime();
    printf("Serial Time: %f seconds\n",e-s);

    s=omp_get_wtime();
    #pragma omp parallel for
    for(int i=1;i<=n;i++)b[i]=prime(i);
    e=omp_get_wtime();
    printf("Parallel Time: %f seconds\n",e-s);
}
```

## Compile

```
gcc -fopenmp prg4.c -o prg4
```

## Run

```
export OMP_NUM_THREADS=4 && ./prg4
```

**Important:** Ubuntu/Linux. Example input 200 prints the primes and both execution times.

# Program 5 – MPI Send and Receive

**Question:** Write an MPI program to demonstrate point-to-point communication using MPI_Send and MPI_Recv.

**Environment:** Ubuntu/Linux with OpenMPI.

## Code

```
#include <stdio.h>
#include <mpi.h>

int main(int argc,char *argv[])
{
    int rank,n=100;

    MPI_Init(&argc,&argv);
    MPI_Comm_rank(MPI_COMM_WORLD,&rank);

    if(rank==0){
        printf("Process 0 sending number %d to process 1\n",n);
        MPI_Send(&n,1,MPI_INT,1,0,MPI_COMM_WORLD);
    }
    else if(rank==1){
        MPI_Recv(&n,1,MPI_INT,0,0,MPI_COMM_WORLD,MPI_STATUS_IGNORE);
        printf("Process 1 received number %d from process 0\n",n);
    }

    MPI_Finalize();
}
```

## Compile

```
mpicc prg5.c -o prg5
```

## Run

```
mpirun --oversubscribe -np 2 ./prg5
```

**Important:** OpenMPI. Run with two processes. Process 0 sends 100 to process 1.

# Program 6 – MPI Deadlock and Avoidance

**Question:** Write an MPI program to demonstrate deadlock between two processes and show how the deadlock can be avoided by ordering communication.

**Environment:** Ubuntu/Linux with OpenMPI.

## Code

```
#include <stdio.h>
#include <mpi.h>

#define MODE 2

int main(int argc,char *argv[])
{
    int rank,n=123;
    MPI_Init(&argc,&argv);
    MPI_Comm_rank(MPI_COMM_WORLD,&rank);

    if(MODE==1){
        printf("Process %d waiting to receive from Process %d...\n",rank,1-rank);
        MPI_Recv(&n,1,MPI_INT,1-rank,0,MPI_COMM_WORLD,MPI_STATUS_IGNORE);
        MPI_Send(&n,1,MPI_INT,1-rank,0,MPI_COMM_WORLD);
    }
    else if(rank==0){
        MPI_Send(&n,1,MPI_INT,1,0,MPI_COMM_WORLD);
        MPI_Recv(&n,1,MPI_INT,1,0,MPI_COMM_WORLD,MPI_STATUS_IGNORE);
        printf("Process 0 received back number: %d\n",n);
    }
    else{
        MPI_Recv(&n,1,MPI_INT,0,0,MPI_COMM_WORLD,MPI_STATUS_IGNORE);
        MPI_Send(&n,1,MPI_INT,0,0,MPI_COMM_WORLD);
        printf("Process 1 received and sent back number: %d\n",n);
    }

    MPI_Finalize();
}
```

## Compile

```
mpicc prg6.c -o prg6
```

## Run

```
mpirun --oversubscribe -np 2 ./prg6
```

**Important:** MODE 2 completes safely. MODE 1 intentionally deadlocks; terminate that run with Ctrl+C.

# Program 7 – MPI Broadcast

**Question:** Write an MPI program to demonstrate broadcasting a value from the root process to all other processes using MPI_Bcast.

**Environment:** Ubuntu/Linux with OpenMPI.

## Code

```
#include <stdio.h>
#include <mpi.h>

int main(int argc,char *argv[])
{
    int rank,n;

    MPI_Init(&argc,&argv);
    MPI_Comm_rank(MPI_COMM_WORLD,&rank);

    if(rank==0){
        n=50;
        printf("Process 0 broadcasting number 50 to all other processes.\n");
    }

    MPI_Bcast(&n,1,MPI_INT,0,MPI_COMM_WORLD);
    printf("Process %d received number %d\n",rank,n);

    MPI_Finalize();
}
```

## Compile

```
mpicc prg7.c -o prg7
```

## Run

```
mpirun --oversubscribe -np 4 ./prg7
```

**Important:** Process 0 initializes 50 and MPI_Bcast sends it to all processes.

# Program 8 – MPI Scatter and Gather

**Question:** Write an MPI program to demonstrate MPI_Scatter and MPI_Gather.

**Environment:** Ubuntu/Linux with OpenMPI.

## Code

```
#include <stdio.h>
#include <mpi.h>

int main(int argc,char *argv[])
{
    int rank,size,x,a[100],g[100];

    MPI_Init(&argc,&argv);
    MPI_Comm_rank(MPI_COMM_WORLD,&rank);
    MPI_Comm_size(MPI_COMM_WORLD,&size);

    if(rank==0)
        for(int i=0;i<size;i++)a[i]=i*10;

    MPI_Scatter(a,1,MPI_INT,&x,1,MPI_INT,0,MPI_COMM_WORLD);
    x+=rank;
    MPI_Gather(&x,1,MPI_INT,g,1,MPI_INT,0,MPI_COMM_WORLD);

    if(rank==0){
        printf("Gathered data in root process:\n");
        for(int i=0;i<size;i++)
            printf("gathered_data[%d] = %d\n",i,g[i]);
    }

    MPI_Finalize();
}
```

## Compile

```
mpicc prg8.c -o prg8
```

## Run

```
mpirun --oversubscribe -np 4 ./prg8
```

**Important:** With four processes, the gathered values are 0, 11, 22, 33.

# Program 9 – MPI Reduce and Allreduce

**Question:** Write an MPI program to demonstrate MPI_Reduce and MPI_Allreduce for sum, product, maximum and minimum.

**Environment:** Ubuntu/Linux with OpenMPI.

## Code

```
#include <stdio.h>
#include <mpi.h>

int main(int argc,char *argv[])
{
    int rank,v,s,p,ma,mi,as,ap,am,ai;

    MPI_Init(&argc,&argv);
    MPI_Comm_rank(MPI_COMM_WORLD,&rank);
    v=rank+1;

    MPI_Reduce(&v,&s,1,MPI_INT,MPI_SUM,0,MPI_COMM_WORLD);
    MPI_Reduce(&v,&p,1,MPI_INT,MPI_PROD,0,MPI_COMM_WORLD);
    MPI_Reduce(&v,&ma,1,MPI_INT,MPI_MAX,0,MPI_COMM_WORLD);
    MPI_Reduce(&v,&mi,1,MPI_INT,MPI_MIN,0,MPI_COMM_WORLD);

    MPI_Allreduce(&v,&as,1,MPI_INT,MPI_SUM,MPI_COMM_WORLD);
    MPI_Allreduce(&v,&ap,1,MPI_INT,MPI_PROD,MPI_COMM_WORLD);
    MPI_Allreduce(&v,&am,1,MPI_INT,MPI_MAX,MPI_COMM_WORLD);
    MPI_Allreduce(&v,&ai,1,MPI_INT,MPI_MIN,MPI_COMM_WORLD);

    if(rank==0){
        printf("--- Results using MPI_Reduce (only root prints) ---\n");
        printf("Sum = %d\nProduct = %d\nMax = %d\nMin = %d\n",s,p,ma,mi);
    }

    printf("Process %d has value %d\n",rank,v);
    printf("Process %d sees (MPI_Allreduce): Sum=%d, Prod=%d, Max=%d, Min=%d\n",
           rank,as,ap,am,ai);

    MPI_Finalize();
}
```

## Compile

```
mpicc prg9.c -o prg9
```

## Run

```
mpirun --oversubscribe -np 4 ./prg9
```

**Important:** With four processes, values are 1–4; sum=10, product=24, max=4 and min=1.